Chapter 2: Probability, Entropy, and Inference

Solutions and comments by C Jones

2.18

Taking the log posterior ratio is a neat trick to get rid of the (often difficult to compute) normalising constant $p(y)$. Bayes' rule give us

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{p(y)}, \quad \text{thus}$$

$$\frac{p(x=1 \mid y)}{p(x=0 \mid y)} = \frac{p(y \mid x=1)p(x=1)}{p(y \mid x=0)p(x=0)}$$

$$\log \frac{p(x=1 \mid y)}{p(x=0 \mid y)} = \log \frac{p(y \mid x=1)}{p(y \mid x=0)} + \log \frac{p(x=1)}{p(x=0)}.$$

2.22

The law of the unconscious statistician (for discrete variables) gives us

$$\mathbb{E}(f(x)) = \sum_{x \in \mathbb{S}(X)} p(x)f(x),$$

where $\mathbb{S}(X)$ is the support of X. Thus, for $f(x) = \frac{1}{p(x)}$, we get

$$\mathbb{E}\left(\frac{1}{p(x)}\right) = \sum_{x \in \mathbb{S}(X)} 1 = |\mathbb{S}(X)|.$$

I.e. the cardinality of the support.

2.25

We want to bound the Shannon entropy of a distribution by cardinality of the support. The definition of Shannon entropy is

$$H(X) = -\sum_{x \in \mathbb{S}(X)} p(x) \log p(x) = \mathbb{E}(-\log p(x)),$$

We also have Jensen's inequality, which tells us that $\mathbb{E}(f(x)) \geq f(\mathbb{E}(x))$ for convex f, with the inequality reversed for concave f. Personally, I don't like the expectation notation (I find it confusing and overloaded), so I'm going to rewrite everything in terms of sums

$$\sum_{x \in \mathbb{S}(X)} p(x)f(x) \geq f\left(\sum_{x \in \mathbb{S}(X)} p(x)x\right).$$

Now, we know that log is a concave function and $-\log$ is convex. It's tempting to immediately go for $f(x) = -\log p(x)$, however this

gives us two problems. First, the inequality will point in the wrong way; second, it's not clear how to handle the RHS when f includes the mass function $p(x)$. The solution is to 'refactor' $f(x)$. Let's instead define $f(y) = \log y$ and $g(x) = \frac{1}{p(x)}$. It's straightforward to see that $f(g(x)) = -\log p(x)$. The trick here is to apply Jensen's inequality on the concave f, whilst leaving $g(x)$ inside the sum as a simple change of variables. This is valid because Jensen's has the general form

$$\sum_{x \in \mathbb{S}(X)} p(x) f(g(x)) \leq f\left(\sum_{x \in \mathbb{S}(X)} p(x) g(x)\right)$$

for concave f.

Solving problems with substitutions like this always feels like magic. I'm not sure I know of a principled way to identify them *a priori*. In fact, without the immediate preceding question, I'm not sure I'd have found it at all.

Using our substitution, we get

$$H(X) = \sum_{x \in \mathbb{S}(X)} p(x) \log\left(\frac{1}{p(x)}\right) \leq \log\left(\sum_{x \in \mathbb{S}(X)} p(x) \frac{1}{p(x)}\right).$$

From the result in the previous problem, we may now obtain our bound on the entropy

$$H(X) \leq \log |\mathbb{S}(X)|.$$

Jensen's inequality is an equality iff $y = \frac{1}{p(x)}$ is constant, thus, $H(X) = \log |\mathbb{S}(X)| \iff p(x) = \frac{1}{\mathbb{S}(X)}$ for all x.

2.31

We'll let random vector $\mathbf{X}$ represent the 2D point that the centre of the coin lands on. If we consider a single square on the grid, we may assume that $\mathbf{X} \sim \prod_{i=1}^{2} \mathcal{U}(0, b)$, i.e. $\mathbf{X}$ is uniformly distributed over the square. Now, simply from the geometry, we can see that the edge of the coin will touch the edge of the square if the sampled $\mathbf{x}$ lies outside of an imaginary inner square with side lengths $b - a$.

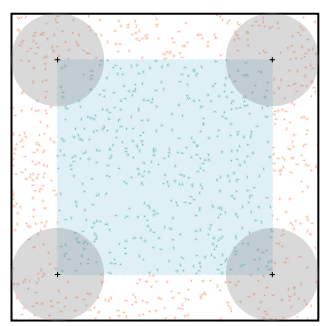

Figure 1: Buffon's coin. If the centre of the coin lands in the blue shaded square, it does not touch the side (as illustrated by the shaded circles).

Since everything is uniform, the problem simply reduces to computing the ratio of the areas of the inner and outer squares

$$p(\text{miss}) = \frac{(b-a)^2}{b^2} = \left(1 - \frac{a}{b}\right)^2.$$

2.32

In Buffon's needle problem, there are two degrees of freedom: vertical location of the centre of the needle y and angle of the needle from horizontal θ. We'll assume that both are uniformly distributed and we will ignore horizontal location because the lines are horizontal. The vertical projection of the needle is $a \sin\theta$.

We'll start with some boundary cases. When $\theta = 0$ (horizontal), the needle has no vertical projection, so a measure zero chance of hitting the horizontal lines. When $\theta = \frac{\pi}{2}$ (vertical), we can use a similar logic to Buffon's coin to see that the needle has a $\frac{a}{b}$ chance of hitting the horizontal lines. Using the sinusoidal vertical projection and these boundary poitns, we can derive

$$p(\text{hit} \mid \theta) = \frac{a}{b} \sin\theta.$$

Now, we may marginalise out θ using the law of total probability

$$p(\text{hit}) = \int_0^{\frac{\pi}{2}} p(\text{hit} \mid \theta) p(\theta) \mathrm{d}\theta = \frac{2}{\pi} \int_0^{\frac{\pi}{2}} \frac{a}{b} \sin\theta \mathrm{d}\theta,$$

where the $\frac{2}{\pi}$ term is the probability density function of the uniform distribution $\mathcal{U}(0, \frac{\pi}{2})$. Solving this integral gives us

$$p(\text{hit}) = -\frac{2a}{b\pi} \left[\cos\theta\right]_0^{\frac{\pi}{2}} = \frac{2a}{b\pi}.$$

If you're not convinced by this logic, you could start by writing out the full conditional $p(\text{hit} \mid y, \theta)$, then marginalising out y to arrive at our $p(\text{hit} \mid \theta)$ expression, i.e.

$$\begin{aligned} p(\text{hit} \mid y, \theta) &= \mathbb{1}_{\{y \leq \frac{a}{2} \sin\theta\}}, \\ p(\text{hit} \mid \theta) &= \int p(\text{hit} \mid y, \theta) p(y \mid \theta) \mathrm{d}y \\ &= \int_0^{\frac{a}{2}\sin\theta} \frac{2}{b} \mathrm{d}y \\ &= a/b \sin\theta. \end{aligned}$$

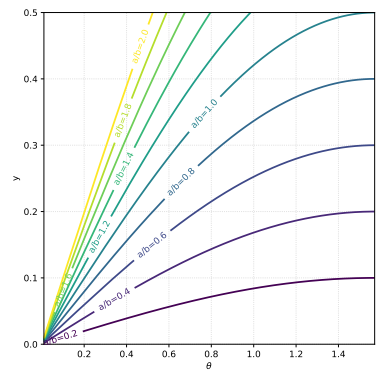

Figure 2: Visualising $p(\text{hit} \mid y, \theta)$ in Buffon's needle. Contours represent boundaries for different $\frac{a}{b}$ lengths. Any configuration below the contour hits the line, any configuration above the contour misses.

2.39

Zipf's law gives us an approximate probability mass function over single English words. For the x'th most common word, $p(x) = \mathbb{1}_{x \in [1, 12368)} \frac{1}{10x}$. We can now compute the entropy of English with the following Python program:

```
import math

def english_pmf(x):
    """Zipf's law of word frequencies in English"""
    if 1 <= x <= 12_367:
        return 0.1 / x
    return 0.0

def english_entropy():
    support = range(1, 12_367 + 1)

    def inner(x):
        p = english_pmf(x)
        return p * -math.log2(p)

    return sum(map(inner, support))

print(english_entropy())
```

This gives an estimated entropy of ≈ 9.7 bits.